

SeedHammer II — the pathological wallet

Backing up the constellation's **pathological example** from files: 11 keys, all four Bitcoin timelock kinds, and a hashlock. Host CLIs, the firmware GUI in the emulator, and the plates. No NFC anywhere.

Test material only — never put funds behind these keys.

The three masters are BIP-39's own published test vectors (abandon...about, legal winner...yellow, letter advice...above). They are public by construction.

The wallet

From mnemonic-toolkit's Examples §5, "Custom degrading-miniscript wallet — **the pathological example**". A four-tier vault that deliberately mixes every timelock kind Bitcoin has:

Tier	Spend condition	Timelock	Keys
1	3-of-3 + secret word	after(1000000) — absolute height	@0 @1 @2
2	2-of-3 + secret word	after(1893456000) — absolute time (2030-01-01)	@3 @4 @5
3	both	older(65535) — relative blocks (~455 d)	@6 @7
4	any 1 of 3	older(4255898) — relative time (~365 d)	@8 @9 @10

The secret word is openseesame; the descriptor commits to $H = \text{sha256}(\text{sha256}(\text{word}))$, shared by tiers 1 and 2. Reusing a *hash* across tiers is fine — it is not a key. The 11 keys come from three masters at divergent account indices (48' / 0' / 0' . . 3' / 2'), so no key is reused.

This is the case that actually forces chunking.

The 12-key multi(5,...) policy tried first encodes to 13 bytes and one md1 string — a keyless BIP-388 template does not pay per key. This one carries two 32-byte hash literals and four timelock arguments, and comes to **182 data symbols against a single-string cap of 80**. It is the first policy in this work that genuinely needs a chunk set.

Toolchain

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md --version
md 0.13.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk --version
mk 0.13.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms --version
ms 0.16.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --version
me 0.7.0
[exit 0]

$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me-preview --version
me-preview 0.7.0
[exit 0]
```

The input files

THE POLICY

inputs-pathological/wallet-policy.txt

```
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(3,@0/<0;1>*,@1/<0;1>*,@2/<0;1>*/))),or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad),multi(2,@3/<0;1>*,@4/<0;1>*,@5/<0;1>*/))),or_i(and_v(v:older(65535),multi(2,@6/<0;1>*,@7/<0;1>*/)),and_v(v:older(4255898),multi(1,@8/<0;1>*,@9/<0;1>*,@10/<0;1>*/))))))
```

THE THREE MASTERS (SECRET — BIP-39 TEST VECTORS)

inputs-pathological/seeds/master-A.seed

abandon abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon about

inputs-pathological/seeds/master-B.seed

legal winner thank year wave sausage worth useful legal win
ner thank yellow

inputs-pathological/seeds/master-C.seed

letter advice cage absurd amount doctor acoustic avoid lett
er advice cage above

THE ELEVEN KEYS

inputs-pathological/keys/key-00.xpub

```
# @0 – tier 1 (3-of-3, after height 1000000 + secret word)
# master A, origin [73c5da0a/48'/0'/0'/2']
xpub6DkFAXwQ2dHxq2vatrt9qyA3bXYU4ToWQwChbf5XB2mSTexchZCeKS1
VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUFL4r6KFrF
```

inputs-pathological/keys/key-01.xpub

```
# @1 – tier 1
# master A, origin [73c5da0a/48'/0'/1'/2']
xpub6DzhyrnFFYQ1HiMDiM388xHnDiRPNdZJFBmmxge3Y1WwChLTMJLfRuh
RHqnQCPbTj3fGKTuKFLHzwpJkp5Dtc3UtLKZKaVZe1yqMBXd6VK
```

inputs-pathological/keys/key-02.xpub

```
# @2 – tier 1
# master A, origin [73c5da0a/48'/0'/2'/2']
xpub6EGx8sPr9FxPPE1rbZazhQWpMXA3Hf5DYKtZbL7c4B5ddzmQktP96U
aTvecEkoCZysuaj79GMCFZYT1KKk7Ph2M3Kf5g8B82KZ8Tz9SKQR
```

inputs-pathological/keys/key-03.xpub

```
# @3 – tier 2 (2-of-3, after time 1893456000 + secret word)
# master A, origin [73c5da0a/48'/0'/3'/2']
xpub6E6Z3S5TXJYNJp4U1q3NZ3pCn82i7KXQAKUtNnzLJ3cCdchQeSdFvX
emizaHUF7wNwRQAB8mPdoZhGHLiv49cWPTCnoJY3Az3E8JKxH9Mq
```

inputs-pathological/keys/key-04.xpub

```
# @4 – tier 2
# master B, origin [b8688df1/48'/0'/0'/2']
xpub6EQya7zGhR92kacYsNnjreouvnHJMpXYsUXnW6NJJAJRCKsa26TzDy4
LdnGhEurrd3d6y1J8PJ7EEMKQp74XTqYvmGJNogYXSKDsZyHtF8mX
```

inputs-pathological/keys/key-05.xpub

```
# @5 – tier 2
# master B, origin [b8688df1/48'/0'/1'/2']
xpub6EAMBjLn1jiQuajTsNRkZXU1oKnA4WjMNvcz4FRR4QmFKdfHxJVvFRL
oysWfcc16AMTR4CoMD8UNjvs9JtbsLeuLwpTczgq8zueERnp8YZF
```

inputs-pathological/keys/key-06.xpub

```
# @6 – tier 3 (both, older 65535 blocks)
# master B, origin [b8688df1/48'/0'/2'/2']
xpub6EGceQV5wHrRemjYkYkRWQefTvsNTw1Lr7JUUp0TgaaWz2P6Cg75o3L
ktGoyMPshKbCqrgy2RXxAmkqBXkbyqPe52CnwA1LUnyBkNUuMhRt
```

inputs-pathological/keys/key-07.xpub

```
# @7 – tier 3
# master B, origin [b8688df1/48'/0'/3'/2']
xpub6ELyn7moeEZ9hJ1HmSm6G5hAfgcCeTa71X9PzYJ39tHNUj93e9MaKb2
tjAFrMFb94NMMeTG7MW8dwxuhhnPhn5swY6r5dxZH6cyiuPd25AQ5
```

inputs-pathological/keys/key-08.xpub

```
# @8 – tier 4 (any 1 of 3, older 4255898 = ~365d)
# master C, origin [28645006/48'/0'/0'/2']
xpub6DnEBNkSJKBYQmsbhS1sP9cNdtU5c9PLFGcjTJmxixc13WB8zNNGQa
zabQpyFAGW5bV9tMko4uBxDxjUKL6dSAcx1tEbgEHTgSqyRsekh6
```

inputs-pathological/keys/key-09.xpub

```
# @9 – tier 4
# master C, origin [28645006/48'/0'/1'/2']
xpub6F6gx8ZP9R3R3eYsU2PeS5EPJ4jN7Wbt9uwyHNXL0aJxQjNT92FGAfC
NDjUDRChWzjfgDuqAZ7Gk9SugPRMa6A8PnzLVvnyEKBW9jHRGRp
```

inputs-pathological/keys/key-10.xpub

```
# @10 – tier 4
# master C, origin [28645006/48'/0'/2'/2']
xpub6Dh4twkBRkzxDSau8FdhVY16jARGnPtfDRUK6LbePrGUGok6xE8SvGt
aygCMLV7beB5YtzkR9dJoic9zYbYy3C7EKc2FGLkKoPT4mQUjPg
```

Host step 1 — it does not fit one string

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/wallet-policy.txt
wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))) ,or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))) ,or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))) ,and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
[exit 0]

$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))) ,or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))) ,or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))) ,and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
md: codec error: payload is 182 data symbols; the codex32 regular code caps single strings at 80 (use chunked encoding / --force-chunked)
[exit 1]
```

Host step 2 — so it is chunked, with an explicit origin

Encoded without `--path`, this policy's card carries no origin and `md` says so: its `wsh(or_i(...))` wrapper has no canonical default derivation path, so `md decode` only PARTIAL-decodes (exit 4, VERIFY-ME) and `me bundle` rejects the set outright. Supplying the origin the warning asks for is what makes the rest of the chain work.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode --group-size 0 --force-chunked --path bip48 wsh(or_i
(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),multi(3,@0/<0;1>/*,
@1/<0;1>/*,@2/<0;1>/*))) ,or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62db
d829b9a08ad)),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))) ,or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<0;1>/*))) ,and_v(v:o
lder(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
chunk-set-id: 0x829c6
md1fs2wxppqzjtvyy4qqxpx2v6mq85yszv6a4pxuusy2unu8xqz8naa2r2akwukvgm42gws92gdhnxq3mrt4
md1fs2wxppqwxk9k7c9xu6pz3ssspyefntdcdhkyqfntk5ymnjqjatj0sucqg70h4gdtkemjesdfjpw8z4kp2ay
md1fs2wxppq3rw4fp6rc6cckmmq5mngy26xpz3t9xdwqq8lluvzze6v6uqpq0pxscq2y6qqh8tw3xyv8tr5s
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

What `--path` does and does not say.

It records ONE shared origin — `bip48` and `m/48'/0'/0'/2'` produce a byte-identical chunk set here. These eleven keys actually sit at four account indices (`48'/0'/0'..3'/2'`) across three masters, and each `mk1` card carries its own true `origin_path`. So the `md1`'s shared value is a default the key cards override, and a restore that trusted the descriptor card alone would derive the wrong key for most slots.

That is now pinned, and the pin corrected the claim.

This paragraph used to say the wrong keys were `@3-@10` and that the case was "worth pinning with a restore test". It has been pinned — `restore_test_pathological.py`, run below — and the measured answer is `@1, @2, @3, @5, @6, @7, @9, @10`: eight of eleven, and the range was wrong in *both* directions. It included `@4` and `@8`, which recover fine, and omitted `@1` and `@2`, which do not. Only `@0, @4 and @8` — one per master, each the account-0 key — come back.

Host step 3 — the chunk set decodes back

Three cards, reassembled, give the same 11-key policy back. **That is a transcription check, not a restore test** — it proves the bytes survive the card format and says nothing about whether the origins they carry are the ones the keys actually live at. The restore test is the next page, and this wallet fails it.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fs2wxppqzjtvyy4qqxpx2v6mq85yszv6a4pxuusy2unu8
xqz8naa2r2akwukvgm42gws92gdhnxq3mrt4 md1fs2wxppqwxk9k7c9xu6pz3ssspyefntdcdhkyqfntk5ymnjqjatj0sucqg70h4gdtkemjesdfjpw8z4kp2
ay md1fs2wxppq3rw4fp6rc6cckmmq5mngy26xpz3t9xdwqq8lluvzze6v6uqpq0pxscq2y6qqh8tw3xyv8tr5s
template: wsh(or_i(and_v(v:after(1000000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b9662375521d0f1ac62dbd829b9a08ad)),m
ulti(3,@0/<0;1>/*,@1/<0;1>/*,@2/<0;1>/*))) ,or_i(and_v(v:after(1893456000),and_v(v:sha256(a84dce40975727c398023cfbd50d5db3b96
62375521d0f1ac62dbd829b9a08ad)),multi(2,@3/<0;1>/*,@4/<0;1>/*,@5/<0;1>/*))) ,or_i(and_v(v:older(65535),multi(2,@6/<0;1>/*,@7/<
0;1>/*))) ,and_v(v:older(4255898),multi(1,@8/<0;1>/*,@9/<0;1>/*,@10/<0;1>/*))))))
n: 11
wallet-policy-mode: false
md1-encoding-id: 829c6ddb2f65a7f47078dd60d5a57ce7
```

wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc

origins:

@0: m/48'/0'/0'/2'

@1: m/48'/0'/0'/2'

@2: m/48'/0'/0'/2'

@3: m/48'/0'/0'/2'

@4: m/48'/0'/0'/2'

@5: m/48'/0'/0'/2'

@6: m/48'/0'/0'/2'

@7: m/48'/0'/0'/2'

@8: m/48'/0'/0'/2'

... clipped; full text in out/transcript.txt

Obstacle 1 — mk cannot derive the policy stub from a chunked md1

Every mk1 key card must carry a 4-byte policy-id stub; `mk encode` refuses without one. The only automatic way to get it is `--from-md1`, and that path rejects a chunk:

```
$ /scratch/code/shibboleth/mnemonic-key/target/release/mk encode --xpub xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTexc
HZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyUFL4r6KFrF --origin-fingerprint 73c5da0a --origin-path m/48'/0'/0'/2' --fro
m-md1 md1fs2wxppqzjtvyvy4qqxpx2v6mq85yszv6a4pxuusyh2unu8xqz8naa2r2akwukvgm42gWSC92gdhnxq3mrt4 --group-size 0
error: md1 input rejected: wire-format version mismatch: got 9, expected 4
[exit 2]
```

`mk vendors md-codec 0.34.0`; the primary crate is at `0.42.0`. The "version 9" it will not parse is the chunked wire form.

WORKING AROUND IT — AND A SPEC/IMPLEMENTATION DIVERGENCE FOUND ON THE WAY

The stub is a property of the *descriptor*, not of the md1 string, so it is well defined for a chunked policy; `md inspect` prints the identities. But which one? `SPEC_mk_v0_1.md §3.3` is explicit — "the top 4 bytes of the MD-encoded policy's **WalletPolicyId**" — and today's `md` prints a field by exactly that name. **That is not the one mk uses**. Measured on a single-string wallet where `--from-md1` does work:

```
wallet-descriptor-template-id: 726a666305756435b7c52c5b3fc69c41
wallet-policy-id: f05e8a1c282f7740bbfd902a759b5577
policy_id_stubs (what mk embedded): 726a6663
```

The stub tracks the **template-id**, not the `wallet-policy-id`. Most likely a rename across versions — `mk vendors md-codec 0.34.0` and the primary is `0.42.0`, which now exposes both — but as it stands the spec sentence and the binary disagree about which identity a key card indexes.

```
$ bash -c /scratch/code/shibboleth/descriptor-mnemonic/target/release/md inspect md1fs2wxppqzjtvyvy4qqxpx2v6mq85yszv6a4pxuu
syh2unu8xqz8naa2r2akwukvgm42gWSC92gdhnxq3mrt4 md1fs2wxpqwxkx9k7c9xu6pzk3ssspyefntdcdhkyqfntk5ymnjqjatj0sucqg70h4gdtkemjesdfj
pw8z4kp2ay md1fs2wxppq3rw4fp6rc6cckmmq5mngy26xpzx3t9xdwqq8lluvzze6v6uqqq0pxscq2y6qqh8tw3xyv8tr5s 2>/dev/null | grep -E 'pol
icy-id|template-id'
wallet-descriptor-template-id: 5b48af35d4321a3ac18b43045e2523cc
wallet-policy-id: f89e23f13c697ae62ef10328d71d7e24
wallet-policy-id-fingerprint: 0xf89e23f1
[exit 0]
```

Following the binary rather than the sentence, this wallet's stub is **5b48af35**, supplied with `--policy-id-stub`.

Host step 4 — the eleven key cards

```
note: 11 key cards -> 30 mk1 chunks (BIP-48 origins push most cards to 3)
$ wc -l /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/card-index.txt
30 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/card-index.txt
[exit 0]

$ wc -l /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/mk-encode-raw.txt
30 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/mk-encode-raw.txt
[exit 0]

$ /scratch/code/shibboleth/mnemonic-key/target/release/mk decode mk1qpd8cwpqqsq4kj90x4eutks2q5zg3vs7rnefw94m5rru59s2su80aw2q
4wgdpapgfl4pkhsdytkwl5z8lphut2hvvpp5av5muuc0cmfrjw2 mk1qpd8cwp806lhaeh6reknylagmwyjycf8044xtt9flsdlkvt6f6cthyL995Lpm5zLrp6
yv6kc36tw
xpub:                xpub6DkFAXWQ2dHxq2vatrt9qyA3bXYU4ToWQwCHbf5XB2mSTexchZCeKS1VZYcPoBd5X8yVcbXFHJR9R8UCVpt82VX1VhR28mCyxUF
L4r6KFrF
origin_fingerprint:  73c5da0a
origin_path:         48'/0'/0'/2'
policy_id_stubs:     5b48af35
chunks:              2 (long)
note: stdout is watch-only — public keys only, cannot spend
[exit 0]
```

Each key splits into **2 chunks**, so the eleven cards are 22 strings. Note the decode: the card carries the origin fingerprint and path, so the origins the descriptor card lacks are present in the bundle.

Host step 5 — the bundle validates, and names every plate

Every set verifies, the manifest emits, and the operator gets the number that matters before touching the machine.

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me bundle --in /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/backup-strings.txt --preview /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/plates --png --manifest /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/manifest.json
me: rendered plate 1 ... (2-25 elided) ... → /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/plates/plate-1.png
me: wrote manifest to /scratch/code/shibboleth/mnemonic-engrave/design/journeys/out/pathological/manifest.json
me: backup needs 34 plates (33 public + ms1 on device):
  plate 1/34  md1 policy → push via NFC & engrave
  plate 2/34  md1 policy → push via NFC & engrave
  plate 3/34  md1 policy → push via NFC & engrave
  plate 4/34  mk1 [b8688df1/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 5/34  mk1 [b8688df1/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 6/34  mk1 [b8688df1/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
  plate 7/34  mk1 [73c5da0a/48'/0'/3'/2'] chunk 1/3 → push via NFC & engrave
  plate 8/34  mk1 [73c5da0a/48'/0'/3'/2'] chunk 2/3 → push via NFC & engrave
  plate 9/34  mk1 [73c5da0a/48'/0'/3'/2'] chunk 3/3 → push via NFC & engrave
  plate 10/34 mk1 [73c5da0a/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 11/34 mk1 [73c5da0a/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 12/34 mk1 [73c5da0a/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 13/34 mk1 [73c5da0a/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 14/34 mk1 [73c5da0a/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 15/34 mk1 [28645006/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 16/34 mk1 [28645006/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 17/34 mk1 [28645006/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 18/34 mk1 [28645006/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 19/34 mk1 [28645006/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 20/34 mk1 [73c5da0a/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 21/34 mk1 [73c5da0a/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 22/34 mk1 [73c5da0a/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
  plate 23/34 mk1 [b8688df1/48'/0'/2'/2'] chunk 1/3 → push via NFC & engrave
  plate 24/34 mk1 [b8688df1/48'/0'/2'/2'] chunk 2/3 → push via NFC & engrave
  plate 25/34 mk1 [b8688df1/48'/0'/2'/2'] chunk 3/3 → push via NFC & engrave
  plate 26/34 mk1 [b8688df1/48'/0'/3'/2'] chunk 1/3 → push via NFC & engrave
  plate 27/34 mk1 [b8688df1/48'/0'/3'/2'] chunk 2/3 → push via NFC & engrave
  plate 28/34 mk1 [b8688df1/48'/0'/3'/2'] chunk 3/3 → push via NFC & engrave
  plate 29/34 mk1 [b8688df1/48'/0'/0'/2'] chunk 1/2 → push via NFC & engrave
  plate 30/34 mk1 [b8688df1/48'/0'/0'/2'] chunk 2/2 → push via NFC & engrave
  plate 31/34 mk1 [28645006/48'/0'/1'/2'] chunk 1/3 → push via NFC & engrave
  plate 32/34 mk1 [28645006/48'/0'/1'/2'] chunk 2/3 → push via NFC & engrave
  plate 33/34 mk1 [28645006/48'/0'/1'/2'] chunk 3/3 → push via NFC & engrave
  plate 34/34 ms1 secret → TYPE ON DEVICE (New > Input Seed > CODEX32); never via this tool
[exit 0]
```


Host step 6 — the seed, and the refusal that still holds

The addresses — the check this wallet could never do

Until the eleven keys were re-derived at BIP-48's four-level origin, md refused every one of them (expected depth 4 for this script context, got 3) and **no address could be derived for this wallet by any tool**. That is why no earlier version of this document showed one. These are the structural half's counterpart: proof the bytes mean the wallet that was intended.

Item 5: the eleven keys used to sit at BIP-84's three-level single-sig account path, while wsh(<miniscript>) is a MultiSig script context that requires a depth-FOUR xpub. md refused every one of them, so NO ADDRESS could be derived for this wallet by any tool -- which is why no journey ever showed one. Re-derived at m/48'/0'/N'/2', they compose.

This is the FUNCTIONAL half of a round trip: the structural half proves the bytes survived, and this proves they mean the wallet we intended. Compare these against your coordinator before engraving anything.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1fatrz2spzjtvyvyy4qqxpx2v6mq85yszv6a4pxuusy2unu8
xqz8naa2r2akwukvgm42gwsr8ztzkypux5za md1fatrz2s2rc6cckmmq5mngy26xzqqyn9xddhpk7cspxdw6snwvgzt4wf7rnqpre774p4wmxytldgm8whvtd8 m
d1fatrz2s4evc3h25sapudvvtas2de5z9drq3rg4jnxhqqqrl7xppvaxdwqqs8sngvq9zvd5z6s7pygwfk3 md1fatrz2su9k6zqhswv0jskp2rsal4egz4ep5
859p875x67p5s3wem7sglxl3d2a3syx3m74wyswrxvcpc54 md1fatrz23zlhah6reknylagmwyjycf8044xtt9fslskvt6f6cthy9yvevrz28llsnr4qy50
0rkxyfe6hw0 md1fatrz23tkxgeal5wa4wy86389z3gq870g9n6rjwuq7cmsyjgffgkqpmffa5k8pmzr2lxjzdetr8v52fs2cv md1fatrz23n04upy3z0n8ek4a
j5kk9satjt7uv4am7jn7q2pvhex36qvqp5csmqq29aj78uss09z0mledqds8p md1fatrz23m7j8mxp87s87gwfzjsys8njyze0uj4852ajv5x7fyu88u23mx7vw
v6azw8d59kw70a2lmlks0w95 md1fatrz2jx88nfhcc7nza0ynr9p5d3exan7jvup3kx8742q6d662qzwwx3qg6rmwy2r0uepgmfdvte0lgjxay md1fatrz2jdk
x6k89kfakaz2az67ajl8hy967je7sks39z80kl7quz6axuvpffejsahhtp2x7gczf3clc26muz md1fatrz2j3w82jq5zap9302ynhp37ggd6z5u4emag0zr8gh9
upnjgr26jfvunvs35jvgdjkvf8nxwprt58vw md1fatrz2jmwxeza6dduqnvtp13vsgcj7vs58sl2hmlrnysxhs57y20d0sn3047m4jnr9cz5e8jumuwp9lg
md1fatrz2n86g9y9evngtkrskaxskvp4qchq5qpv0pez9rarvxl32c8627y2x4unucwuan7ngdj6m8a4jm3a38 md1fatrz2ntm5awv08c2v0vktw5t6qu9g4xkm
v2dywrwkudljyuk9dam643wu0vf30dd15vwq28tgapq6km0dg md1fatrz2n3g59kdh7306na4nhq32cd5w9c2uzlhxryrtzdu2qf455xvp5uc0ktmeyw4h7cayj
9dzdt2xm663g md1fatrz2nuquhu772hljrlwlhlnccfd22zx672chhdu5ajtemtchqtljnzwtntna6j74phutvj10dhs8ws22759 md1fatrz25q9z7s576y0hwv6
a92khwlfwmwulc9zrqupz2zll50ju3dcmghtxtfv0y025ltk2rlu3mjg4q49y md1fatrz25fnqlk8ycusz0quhfev0a3hqdgghjmkzs3ry92d3gv4ejtmu9f0zx
f3clxvtl1nnvxgag09mvs08va3 md1fatrz25hgjc5r8x9t5j54jfadzzykqz37sefnc0rhk5zchwu9cfljgk8tg3nlnyr6nv2e7vtf5ttahr4hn4 md1fatrz25e
fq2e50d8mg50ts8t8mqquwc90uynctuc4sv9py5lug68qthk65sax8thnpjxetx05qdw2xzn9759 md1fatrz24xrn6wz3pd0pwa4xqjj8e7htel64394h6gme2g4
flvkpvqrj3us7k4x9dcp2a2uz2f0tdhlxnxyd7a md1fatrz24w75nz845y9up7y276lt29ywu3usza3aqrp2qarn88u9dtnzuy --chain 0 --count 3
bc1qkuknuyp6dsm0fq44cyhzqy9wl3ex2n6ed39zxhx867l9wlh4yhlsejms64
bc1q6k2emh2epd57p5qfwswnahtphjlzsuc5x92c7zphr2lnnckzrpzqjdara
bc1q97cvjd2wj75hu62w4at6w4ay0mz3janslla3y3rml4lglgym38qfh70pa
note: stdout is watch-only – public keys only, cannot spend
[exit 0]
```

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md address md1fatrz2spzjtvyvyy4qqxpx2v6mq85yszv6a4pxuusy2unu8
xqz8naa2r2akwukvgm42gwsr8ztzkypux5za md1fatrz2s2rc6cckmmq5mngy26xzqqyn9xddhpk7cspxdw6snwvgzt4wf7rnqpre774p4wmxytldgm8whvtd8 m
d1fatrz2s4evc3h25sapudvvtas2de5z9drq3rg4jnxhqqqrl7xppvaxdwqqs8sngvq9zvd5z6s7pygwfk3 md1fatrz2su9k6zqhswv0jskp2rsal4egz4ep5
859p875x67p5s3wem7sglxl3d2a3syx3m74wyswrxvcpc54 md1fatrz23zlhah6reknylagmwyjycf8044xtt9fslskvt6f6cthy9yvevrz28llsnr4qy50
0rkxyfe6hw0 md1fatrz23tkxgeal5wa4wy86389z3gq870g9n6rjwuq7cmsyjgffgkqpmffa5k8pmzr2lxjzdetr8v52fs2cv md1fatrz23n04upy3z0n8ek4a
j5kk9satjt7uv4am7jn7q2pvhex36qvqp5csmqq29aj78uss09z0mledqds8p md1fatrz23m7j8mxp87s87gwfzjsys8njyze0uj4852ajv5x7fyu88u23mx7vw
v6azw8d59kw70a2lmlks0w95 md1fatrz2jx88nfhcc7nza0ynr9p5d3exan7jvup3kx8742q6d662qzwwx3qg6rmwy2r0uepgmfdvte0lgjxay md1fatrz2jdk
x6k89kfakaz2az67ajl8hy967je7sks39z80kl7quz6axuvpffejsahhtp2x7gczf3clc26muz md1fatrz2j3w82jq5zap9302ynhp37ggd6z5u4emag0zr8gh9
upnjgr26jfvunvs35jvgdjkvf8nxwprt58vw md1fatrz2jmwxeza6dduqnvtp13vsgcj7vs58sl2hmlrnysxhs57y20d0sn3047m4jnr9cz5e8jumuwp9lg
md1fatrz2n86g9y9evngtkrskaxskvp4qchq5qpv0pez9rarvxl32c8627y2x4unucwuan7ngdj6m8a4jm3a38 md1fatrz2ntm5awv08c2v0vktw5t6qu9g4xkm
v2dywrwkudljyuk9dam643wu0vf30dd15vwq28tgapq6km0dg md1fatrz2n3g59kdh7306na4nhq32cd5w9c2uzlhxryrtzdu2qf455xvp5uc0ktmeyw4h7cayj
9dzdt2xm663g md1fatrz2nuquhu772hljrlwlhlnccfd22zx672chhdu5ajtemtchqtljnzwtntna6j74phutvj10dhs8ws22759 md1fatrz25q9z7s576y0hwv6
a92khwlfwmwulc9zrqupz2zll50ju3dcmghtxtfv0y025ltk2rlu3mjg4q49y md1fatrz25fnqlk8ycusz0quhfev0a3hqdgghjmkzs3ry92d3gv4ejtmu9f0zx
f3clxvtl1nnvxgag09mvs08va3 md1fatrz25hgjc5r8x9t5j54jfadzzykqz37sefnc0rhk5zchwu9cfljgk8tg3nlnyr6nv2e7vtf5ttahr4hn4 md1fatrz25e
fq2e50d8mg50ts8t8mqquwc90uynctuc4sv9py5lug68qthk65sax8thnpjxetx05qdw2xzn9759 md1fatrz24xrn6wz3pd0pwa4xqjj8e7htel64394h6gme2g4
flvkpvqrj3us7k4x9dcp2a2uz2f0tdhlxnxyd7a md1fatrz24w75nz845y9up7y276lt29ywu3usza3aqrp2qarn88u9dtnzuy --chain 1 --count 3
bc1qxtlrmq23lczrtx8l9x3nsfp3u5h7vt9zjmy94m6srl354ps9amus2q56lx
bc1q6keukt5pnwv0mtr593gj8p4yvpvt5gl4hwhzyxzuy06mtj7leauqyk8qt
bc1qcy83peqe3n6flckwqf3fvr2g0y2z8hplfxfm8nfecaxas4ux2sjjwuld
note: stdout is watch-only – public keys only, cannot spend
[exit 0]
```

The restore test — what a card-only restore recovers

The decode step is a transcription check. **This is the restore test**: it derives, from each master seed, the key the *descriptor card's own declared origin* would lead a restorer to, and compares it against the key that slot actually holds.

```
$ python3 /scratch/code/shibboleth/mnemonic-engrave/design/journeys/restore_test_pathological.py
A restore that trusts the DESCRIPTOR CARD alone derives account 0 from every master.
```

@0	master	73c5da0a	at	48'/0'/0'/2'	RECOVERED
@1	master	73c5da0a	at	48'/0'/1'/2'	WRONG KEY
@2	master	73c5da0a	at	48'/0'/2'/2'	WRONG KEY

```
@3 master 73c5da0a at 48'/0'/3'/2' WRONG KEY
@4 master b8688df1 at 48'/0'/0'/2' RECOVERED
@5 master b8688df1 at 48'/0'/1'/2' WRONG KEY
@6 master b8688df1 at 48'/0'/2'/2' WRONG KEY
@7 master b8688df1 at 48'/0'/3'/2' WRONG KEY
@8 master 28645006 at 48'/0'/0'/2' RECOVERED
@9 master 28645006 at 48'/0'/1'/2' WRONG KEY
@10 master 28645006 at 48'/0'/2'/2' WRONG KEY
```

8 of 11 slots recover the WRONG key: [1, 2, 3, 5, 6, 7, 9, 10]
 3 recover correctly: [0, 4, 8]

Every tier of this vault needs keys this restore cannot produce.

Matches what the journey documents. Gate green.
 [exit 0]

Three of eleven.

Only @0, @4 and @8 — one account-0 key per master — come back. Every tier of this vault needs signatures this restore cannot produce, so the descriptor card alone is not a backup of this wallet: the mk1 key cards carry the origins that make it one, and they are not optional.

```
$ cat /scratch/code/shibboleth/mnemonic-engrave/design/journeys/inputs-pathological/seeds/master-A.seed
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-secret/target/release/ms encode --phrase abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon about
msl0e ntrs qqqqq qqqqq qqqqq qqqqq qqcj9 sxraq 34v7f
word count: 12
language: english (BIP-39 checksum valid)
passphrase: not stored in msl (record separately if used)
warning: stdout carries private key material (can spend) - redirect or encrypt (e.g. '> file.txt' or '| age -e ...')
[exit 0]
```

```
$ /scratch/code/shibboleth/mnemonic-engrave/target/release/me --in /dev/fd/63 --hex
me: refusing to emit msl over NFC: msl is secret seed entropy and must never be transmitted by radio (RF eavesdropping risk). Enter it by hand on the device: New > Input Seed > CODEX32.
[exit 3]
```

The 25 public plates

Three descriptor chunks, then eleven key cards at two chunks each.



plate 1 — md1 policy, chunk 1/3



plate 2 — md1 policy, chunk 2/3



plate 3 — md1 policy, chunk 3/3



plate 4 — @5 [b8688df1/48'/0'/1'/2'] chunk 1/3



plate 5 — @5 [b8688df1/48'/0'/1'/2'] chunk 2/3



plate 6 — @5 [b8688df1/48'/0'/1'/2'] chunk 3/3



plate 7 — @3 [73c5da0a/48'/0'/3'/2'] chunk 1/3



plate 8 — @3 [73c5da0a/48'/0'/3'/2'] chunk 2/3



plate 9 — @3 [73c5da0a/48'/0'/3'/2'] chunk 3/3



plate 10 — @2 [73c5da0a/48'/0'/2'/2'] chunk 1/3



plate 11 — @2 [73c5da0a/48'/0'/2'/2'] chunk 2/3



plate 12 — @2 [73c5da0a/48'/0'/2'/2'] chunk 3/3

The 25 public plates, continued



plate 13 — @0 [73c5da0a/48/0/0/2] chunk 1/2



plate 14 — @0 [73c5da0a/48/0/0/2] chunk 2/2



plate 15 — @8 [28645006/48/0/0/2] chunk 1/2



plate 16 — @8 [28645006/48/0/0/2] chunk 2/2



plate 17 — @10 [28645006/48/0/2/2] chunk 1/3



plate 18 — @10 [28645006/48/0/2/2] chunk 2/3



plate 19 — @10 [28645006/48/0/2/2] chunk 3/3



plate 20 — @1 [73c5da0a/48/0/1/2] chunk 1/3



plate 21 — @1 [73c5da0a/48/0/1/2] chunk 2/3



plate 22 — @1 [73c5da0a/48/0/1/2] chunk 3/3



plate 23 — @6 [b8688df1/48/0/2/2] chunk 1/3



plate 24 — @6 [b8688df1/48/0/2/2] chunk 2/3



plate 25 — @6 [b8688df1/48/0/2/2] chunk 3/3



plate 26 — @7 [b8688df1/48/0/3/2] chunk 1/3



plate 27 — @7 [b8688df1/48/0/3/2] chunk 2/3



plate 28 — @7 [b8688df1/48/0/3/2] chunk 3/3



plate 29 — @4 [b8688df1/48/0/0/2] chunk 1/2



plate 30 — @4 [b8688df1/48/0/0/2] chunk 2/2



plate 31 — @9 [28645006/48/0/1/2] chunk 1/3



plate 32 — @9 [28645006/48/0/1/2] chunk 2/3



plate 33 — @9 [28645006/48/0/1/2] chunk 3/3

Plus three seed plates that are not here.

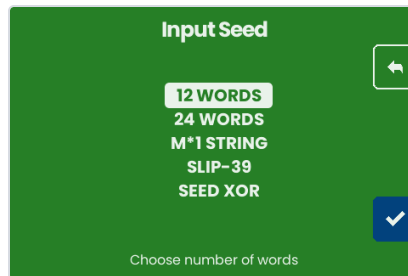
Each master is engraved from words typed on the machine. No host tool will render, transmit or preview them — so the real total is **28 plates.**

On the machine — the seed, typed

No NFC in this journey. The seed goes in the way the machine's own checklist says it must: by hand, on the device.



Boot: Backup Wallet



Input Seed — 12 words



Word 1 of 12



Three letters is enough: "1 match", auto-completed



All 12 accepted — the BIP-39 checksum holds



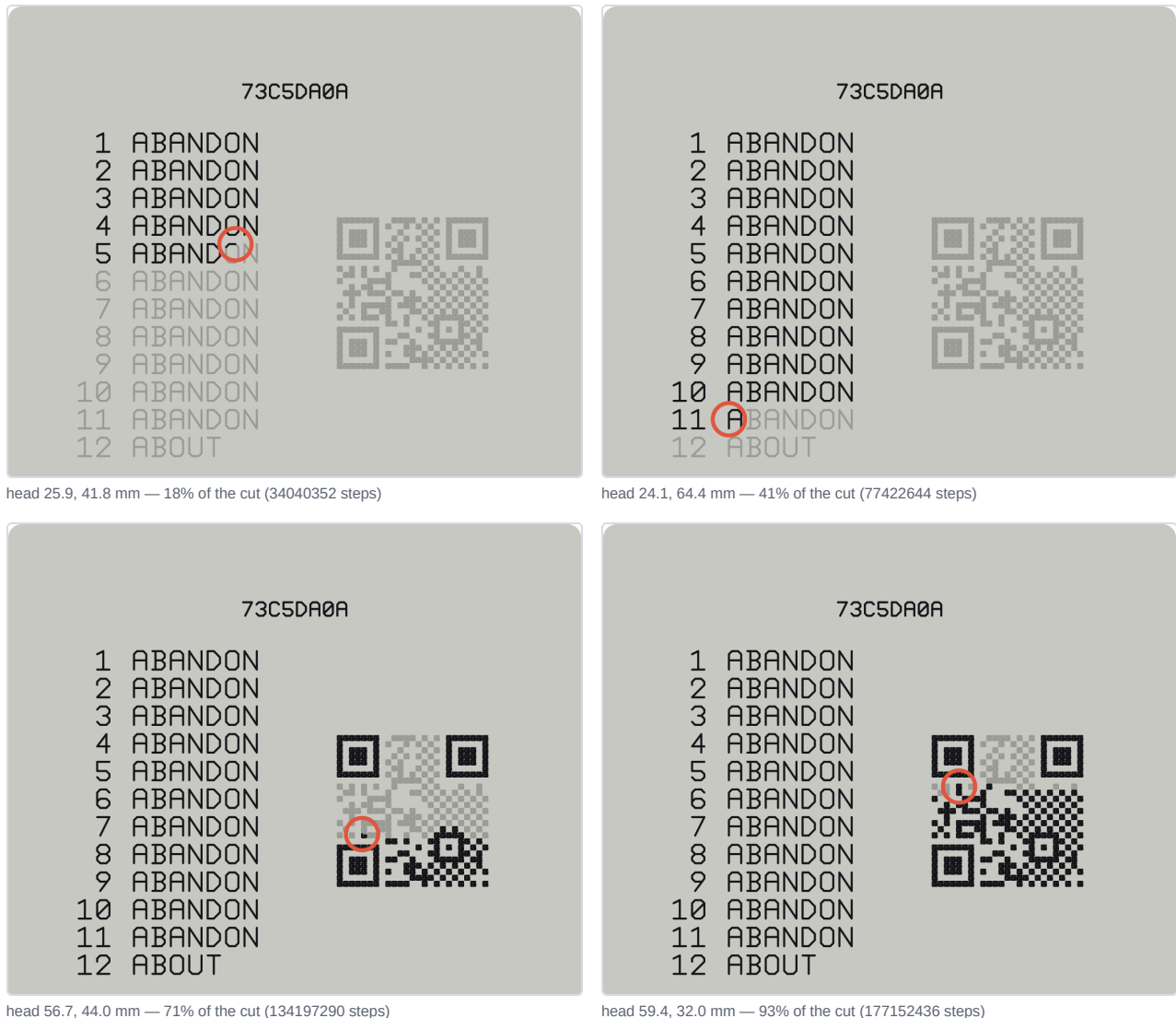
Optional BIP-39 passphrase, skipped

On the machine — the seed plate



The plate overlay, live

The whole planned layout is drawn in **grey** the moment the engrave begins, from the same PlanEngraving call the firmware makes. What the driver has actually stepped is filled in **black**, decoded from the real step stream. The red circle is the head. The title is the master fingerprint 73C5DA0A — the same one the descriptor's origins name.



What this run turned up

1. `mk encode --from-md1` cannot read a chunked md1.

wire-format version mismatch: got 9, expected 4, exit 2 — and a stub is mandatory, so for any policy large enough to need chunking the documented route to a key card does not work. `mk vendors md-codec 0.34.0` against the primary's `0.42.0`; version 9 is the chunked wire form its copy predates. The provenance pin is what should have caught this. Workaround: read the identity out of `md inspect` and pass `--policy-id-stub` by hand — which nothing tells an operator to do, and which requires knowing the next finding.

2. The spec and the binary disagree about which identity the stub indexes.

`SPEC_mk_v0_1.md` §3.3 says the stub is the top 4 bytes of the **WalletPolicyId**, and `md` prints a field of exactly that name. Measured, `mk` embeds the top 4 bytes of the **wallet-descriptor-template-id** instead — `726a6663` where the `wallet-policy-id` began `f05e8a1c`. Probably a rename that landed in `md-codec` after `0.34.0`, but the sentence and the code now name different things, and a stub is the index a recovering operator uses to tell wallets apart.

3. `--path` is required here, and it flattens.

Without it the descriptor card carries no origin, `md decode PARTIAL`-decodes (exit 4) and `me bundle` refuses the set. With it, everything validates — but it records a single shared origin while these eleven keys sit at four account indices. The true paths survive on the `mk1` cards; the descriptor card alone restores only **3 of 11** slots correctly — `@0`, `@4` and `@8`, one account-0 key per master. Measured by the restore test above, which also corrected this sentence's earlier claim of "`@3–@10`". Which source wins is a *design* question rather than a defect, and it is now pinned rather than argued.

4. `md encode` has no per-key origin flag.

`--key` takes a bare xpub and rejects an origin-annotated one (`base58check decode`), and `--fingerprint` carries no path. So a divergent-origin wallet cannot state its origins in the descriptor card at all — only the flattened form of finding 3 is expressible.

5. Carried over from the first journey:

presenting an NFC tag to a gathering flow freezes the emulator (filed as **F-126**). This journey avoids NFC entirely, so it is not exercised here.

Corrected from the first draft of this document.

It reported `me bundle`'s refusal as a tool defect and rendered the plates by calling the sidecar directly. That was wrong: the refusal was the consequence of omitting `--path`, which `md` had warned about at encode time. The plates and checklist here come from `me bundle --preview`.

Reproducing this document

```
cd <this directory>
bash transcript_pathological.sh > transcript_pathological.txt 2>&1 # every CLI block, refusals included
python3 capture_pathological.py                                     # the 13 screenshots, from the emulator
python3 build_pdf_pathological.py                                  # this document, as a PDF
```

Emulator frames were captured by `POSTing canvas.toDataURL()` to a local receiver, so each screenshot is the device framebuffer exactly. Plate overlays are the page's own SVG, rasterised in the browser that drew them. Both are produced by `capture_pathological.py`, which drives `cmd/emu/shots_pathological.js` — the capture used to be console code in a session that no longer exists, which is why this document could not be rebuilt (F-210).